



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Motor de Integridad Multibase NoSQL

Curso: Base de Datos II

Docente: Ing. Patrick José Cuadros Quiroga

Integrantes:

Salas Acuña, Adrian Fabricio (2021069829)

Lopez Ojeda, Keyla Noelia (2023078690)

**Tacna – Perú
2026**



Datastack



CloudNexus

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	KLO / ASA	ASA	KLO	01/10/2026	Versión original



Datastack · CloudNexus S.A.C.

Sistema Motor de Integridad Multibase NoSQL

Documento de Visión

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	KLO / ASA	ASA	KLO	01/10/2026	Versión original

ÍNDICE GENERAL

1. Introducción	4
1.1 Propósito	4
1.2 Alcance.....	4
1.3 Definiciones, Siglas y Abreviaturas.....	5
1.4 Referencias	6
1.5 Visión General	7
2. Posicionamiento.....	7
2.1 Oportunidad de negocio	7
2.2 Definición del problema	8
3. Descripción de los interesados y usuarios	9
3.1 Resumen de los interesados	9
3.2 Resumen de los usuarios	9
3.3 Entorno de usuario	10
3.4 Perfiles de los interesados.....	10
3.5 Perfiles de los Usuarios	11
3.6 Necesidades de los interesados y usuarios	13
4. Vista General del Producto	14
4.1 Perspectiva del producto	14
4.2 Resumen de capacidades	15
4.3 Suposiciones y dependencias	15
4.4 Costos y precios	16
4.5 Licenciamiento e instalación	17
5. Características del producto.....	18
6. Restricciones	19
7. Rangos de calidad	20
8. Precedencia y Prioridad	21
9. Otros requerimientos del producto	21
a) Estándares legales	21
b) Estándares de comunicación	22
c) Estándares de cumplimiento de la plataforma	22
d) Estándares de calidad y seguridad	22
CONCLUSIONES	23
RECOMENDACIONES	23
BIBLIOGRAFIA.....	24
WEBGRAFIA	25

1. Introducción

El presente documento de visión describe, a alto nivel, el sistema *Motor de Integridad Multibase NoSQL*, que la empresa Datastack desarrolla para su cliente CloudNexus S.A.C. Siguiendo la plantilla del Proceso Unificado de Rational (RUP), el documento recoge las necesidades de los interesados y usuarios, la posición del producto frente al problema que resuelve y las características que debe ofrecer, con el fin de establecer un entendimiento común entre el cliente y el equipo de desarrollo antes de detallar los requisitos (Kruchten, 2004).

1.1 Propósito

El propósito de este documento es definir la visión del producto: qué problema resuelve, para quién lo resuelve y cuáles son sus capacidades principales. Sirve como acuerdo entre CloudNexus y Datastack sobre el alcance de la solución y como punto de partida para la especificación de requisitos de software (FD03) y el documento de arquitectura (FD04).

Está dirigido a la gerencia y al área de tecnología de CloudNexus, que validan que la solución responde a sus necesidades; al equipo de Datastack, que lo utiliza como guía de diseño y construcción; y al docente del curso, que evalúa la coherencia entre el problema planteado y la solución propuesta.

1.2 Alcance

El documento abarca la versión 1.0 del Motor de Integridad Multibase NoSQL, una aplicación web desplegada en la nube que permite a cada usuario conectar sus propias bases de datos MongoDB, Redis y Apache Cassandra, definir reglas de integridad sin escribir código y verificar su cumplimiento. El sistema cubre los siguientes procesos:

- Registro e inicio de sesión de usuarios, con contraseñas protegidas mediante *bcrypt*.
- Gestión de las conexiones a los tres motores, con credenciales cifradas por usuario.

- Definición de reglas de cuatro tipos: esquema, unicidad, integridad referencial y consistencia.
- Validación de un registro antes de escribirlo (simulador previo a la escritura) y escaneo por lotes de los datos ya almacenados.
- Monitoreo del estado de cada motor, tablero de métricas, reportes de anomalías en PDF y registro de auditoría centralizado.

Quedan fuera del alcance las transacciones distribuidas entre motores, la corrección automática de los datos que incumplen una regla, la interceptación de escrituras realizadas por aplicaciones de terceros directamente sobre los motores y el soporte de otros motores NoSQL.

1.3 Definiciones, Siglas y Abreviaturas

La Tabla 1 reúne los términos, siglas y abreviaturas que se utilizan a lo largo del documento.

Tabla 1

Definiciones, Siglas y Abreviaturas

Término	Definición
Adaptador	Componente que traduce las operaciones del motor de reglas a las instrucciones propias de cada base de datos (MongoDB, Redis o Cassandra).
Auditoría	Registro histórico de cada violación detectada, con la regla, el motor, la fecha y el dato afectado.
bcrypt	Función de derivación de contraseñas adaptable al costo computacional (Provos y Mazières, 1999).
CAP	Teorema que establece que un sistema distribuido no puede garantizar a la vez consistencia, disponibilidad y tolerancia a particiones (Gilbert y Lynch, 2002).
Consistencia eventual	Modelo en el que las réplicas de un dato convergen con el tiempo, aunque temporalmente puedan diferir (Vogels, 2009).
CQL	<i>Cassandra Query Language</i> , lenguaje de consultas de Apache Cassandra.
Escaneo por lotes	Revisión masiva de los datos ya almacenados para detectar violaciones de las reglas activas (<i>Scan Batch</i>).
Fernet	Esquema de cifrado simétrico autenticado de la biblioteca <i>cryptography</i> de Python.
Integridad referencial	Garantía de que toda referencia entre registros apunte a un registro existente.

Término	Definición
JSON Schema	Vocabulario estándar para describir y validar la estructura de documentos JSON.
MVC	Patrón Modelo-Vista-Controlador, que separa datos, interfaz y lógica de control.
NoSQL	Familia de bases de datos no relacionales: documentales, clave-valor, de columnas anchas y de grafos.
Persistencia políglota	Uso de distintos motores de datos dentro de un mismo sistema según la necesidad de cada componente (Sadalage y Fowler, 2012).
Plano de control	Base de datos que almacena la configuración del sistema: usuarios, reglas, conexiones y auditoría (Supabase).
RLS	<i>Row Level Security</i> , seguridad a nivel de fila de PostgreSQL.
RUP	<i>Rational Unified Process</i> , proceso de desarrollo de software iterativo (Kruchten, 2004).
TLS	<i>Transport Layer Security</i> , protocolo que cifra las comunicaciones en red (Rescorla, 2018).
Violación o anomalía	Registro que incumple una regla de integridad activa.

Nota. Elaboración propia.

1.4 Referencias

Los documentos y normas que complementan esta visión son los siguientes:

Datastack. (2026). *FD01: Informe de factibilidad del proyecto Motor de Integridad Multibase*

NoSQL (Versión 1.0) [Documento de proyecto].

International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC Standard No. 25010:2023).

Kruchten, P. (2004). *The Rational Unified Process: An introduction* (3.^a ed.). Addison-Wesley.

Lopez Ojeda, K. N., y Salas Acuña, A. F. (2026). *Semana 1: Título, problema y objetivos del proyecto “Motor de Integridad Multibase NoSQL”* [Informe académico]. Universidad Privada de Tacna.

Lopez Ojeda, K. N., y Salas Acuña, A. F. (2026). *motor-integridad* [Código fuente]. GitHub.
<https://github.com/keylaloopez/motor-integridad>

1.5 Visión General

El documento se organiza en nueve capítulos. El capítulo 2 presenta el posicionamiento del producto: la oportunidad de negocio y la definición del problema. El capítulo 3 describe a los interesados y usuarios, su entorno y sus necesidades. El capítulo 4 ofrece una vista general del producto, sus capacidades, dependencias, costos y licenciamiento. Los capítulos 5 a 8 detallan las características del producto, sus restricciones, los rangos de calidad esperados y la prioridad de cada característica. El capítulo 9 reúne los estándares legales, de comunicación, de plataforma y de calidad que el producto debe cumplir. Finalmente, se presentan las conclusiones, las recomendaciones y las fuentes consultadas.

2. Posicionamiento

2.1 Oportunidad de negocio

CloudNexus S.A.C. es una empresa tacneña que ofrece a pequeñas y medianas empresas una plataforma de comercio electrónico y logística en la nube. Su arquitectura sigue el enfoque de persistencia políglota: MongoDB almacena el catálogo y los clientes, Redis mantiene en memoria las sesiones y los pedidos en curso, y Apache Cassandra registra el historial de pagos y eventos. Esta combinación es habitual en el mercado: según la encuesta para desarrolladores de Stack Overflow (2025), Redis es utilizado por el 28 % de los encuestados y MongoDB por el 24 %.

Ninguno de estos motores garantiza por sí mismo que los datos sean coherentes entre ellos. MongoDB trabaja con un esquema flexible por defecto (MongoDB, Inc., s.f.-b), Cassandra no tiene claves foráneas ni integridad relacional (The Apache Software Foundation, s.f.) y Redis no declara esquema alguno (Redis Ltd., s.f.). Como resultado, CloudNexus detecta pagos asociados a pedidos inexistentes, clientes duplicados y datos que no coinciden entre motores, y los corrige con scripts manuales que no dejan evidencia auditable. Gartner estima que la mala calidad de los datos cuesta a las organizaciones un promedio de 12,9 millones de dólares al año (Sakpal, 2021).

Existe, por tanto, la oportunidad de ofrecer una herramienta ligera, independiente del motor y desplegada en la nube que centralice las reglas de integridad, las verifique sobre los tres motores y registre cada violación. La Tabla 2 resume la posición del producto.

Tabla 2

Sentencia que Define la Posición del Producto

Para	CloudNexus S.A.C. y, en general, organizaciones que combinan varias bases de datos NoSQL.
Quienes	Necesitan asegurar que sus datos cumplan reglas de esquema, unicidad, integridad referencial y consistencia, aunque los motores no las impongan.
El producto	Motor de Integridad Multibase NoSQL.
Que	Permite definir reglas sin programar, validar registros antes de escribirlos, escanear los datos existentes y auditar las violaciones desde una aplicación web.
A diferencia de	La validación nativa de cada motor, que se limita a una sola base de datos, y de los scripts manuales, que no son reutilizables ni auditables.
Nuestro producto	Aplica las mismas reglas sobre MongoDB, Redis y Cassandra mediante adaptadores, incluidas reglas entre motores, y conserva un historial de auditoría centralizado.

Nota. Formato adaptado de la plantilla de visión de RUP (Kruchten, 2004).

2.2 Definición del problema

El problema central que atiende el producto se resume en la Tabla 3, siguiendo la estructura propuesta por RUP para la definición del problema.

Tabla 3

Definición del Problema

El problema de	La ausencia de mecanismos para definir, verificar y auditar reglas de integridad sobre datos distribuidos en MongoDB, Redis y Apache Cassandra.
Afecta a	Los administradores de bases de datos, los analistas de calidad de datos, los desarrolladores y la gerencia de CloudNexus, así como a los clientes finales de la plataforma.
Cuyo impacto es	Datos huérfanos, registros duplicados y réplicas que divergen entre motores; reportes y facturación incorrectos; horas invertidas en correcciones manuales y falta de trazabilidad ante auditorías.
Una solución exitosa debería	Centralizar las reglas, validarlas antes y después de la escritura en los tres motores, registrar cada violación con su contexto y presentar los resultados en un tablero y en reportes descargables.

Nota. Elaboración propia.

3. Descripción de los interesados y usuarios

Esta sección identifica a las personas y áreas que tienen interés en el producto o que lo utilizan directamente, junto con su entorno de trabajo y sus necesidades.

3.1 Resumen de los interesados

Tabla 4

Resumen de los Interesados

Nombre	Descripción	Responsabilidades
Gerencia General de CloudNexus	Patrocinador del proyecto.	Aprueba el presupuesto, valida el cumplimiento de los objetivos y autoriza la puesta en producción.
Jefatura de Tecnología de CloudNexus	Responsable de la infraestructura de datos de la empresa.	Define los requisitos técnicos, provee el acceso a los motores y acepta los entregables.
Área de Calidad de Datos y Auditoría	Responsable de la exactitud y trazabilidad de la información.	Define las reglas de negocio que deben cumplirse y revisa los reportes de anomalías.
Equipo de proyecto de Datastack	Proveedor de la solución.	Analiza, diseña, construye, prueba, despliega y documenta el sistema; brinda soporte durante la puesta en marcha.

Nota. Elaboración propia.

3.2 Resumen de los usuarios

Tabla 5

Resumen de los Usuarios

Nombre	Descripción	Responsabilidades	Interesado
Administrador de bases de datos	Gestiona los motores MongoDB, Redis y Cassandra.	Registra las conexiones, monitorea los motores y ejecuta los escaneos.	Jefatura de Tecnología
Analista de calidad de datos	Define y mantiene las reglas de integridad.	Crea reglas, ejecuta verificaciones y analiza las anomalías.	Área de Calidad de Datos
Desarrollador de aplicaciones	Programa los servicios que escriben en los motores.	Valida registros en el simulador antes de llevarlos a producción.	Jefatura de Tecnología
Auditor interno	Supervisa el cumplimiento de las políticas de datos.	Consulta el tablero y descarga los reportes de anomalías en PDF.	Gerencia General

Nota. Elaboración propia.

3.3 Entorno de usuario

Se estima que entre cinco y quince usuarios de CloudNexus utilizarán el sistema de forma simultánea. Cada usuario trabaja en su propio espacio: sus conexiones, reglas y resultados quedan asociados a su cuenta, y las credenciales de los motores se almacenan cifradas en el plano de control.

El sistema se usa desde un navegador web actualizado (Google Chrome, Microsoft Edge o Mozilla Firefox), sin instalar software adicional, a través de la dirección pública en Streamlit Community Cloud. La interfaz está en español y organizada en módulos: Conexiones, Reglas, Ejecución, Monitor, Dashboard, Reportes y Comunidad. Una validación individual se completa en segundos, mientras que un escaneo por lotes depende del volumen de datos y suele tomar pocos minutos para colecciones de tamaño medio.

Los motores se encuentran en servicios gestionados en la nube (MongoDB Atlas, Upstash y DataStax Astra), por lo que el usuario necesita conexión a internet y que dichos servicios acepten conexiones desde la aplicación.

3.4 Perfiles de los interesados

Las Tablas 6 a 9 detallan el perfil de cada interesado identificado.

Tabla 6

Perfil del Interesado: Gerencia General de CloudNexus

Descripción	Máxima autoridad de la empresa cliente y patrocinador del proyecto.
Tipo	Ejecutivo; usuario ocasional de los reportes.
Responsabilidades	Aprobar el presupuesto, el cronograma y la puesta en producción.
Criterios de éxito	Reducción de incidencias por datos inconsistentes y evidencia auditable de la calidad de los datos.
Grado de participación	Revisión de hitos y aprobación final.
Comentarios	Requiere indicadores claros y reportes ejecutivos.

Tabla 7

Perfil del Interesado: Jefatura de Tecnología de CloudNexus

Descripción	Responsable de la infraestructura y de las bases de datos de la empresa.
Tipo	Experto técnico.
Responsabilidades	Definir requisitos técnicos, facilitar credenciales de prueba y aceptar entregables.
Criterios de éxito	Integración con los tres motores sin modificar las aplicaciones existentes y con credenciales protegidas.
Grado de participación	Alto: revisa cada iteración.
Comentarios	Prioriza la seguridad y la facilidad de mantenimiento.

Tabla 8

Perfil del Interesado: Área de Calidad de Datos y Auditoría

Descripción	Área que vela por la exactitud, completitud y consistencia de la información.
Tipo	Usuario de negocio.
Responsabilidades	Definir las reglas de negocio y validar los reportes de anomalías.
Criterios de éxito	Detección de todas las violaciones del conjunto de prueba y reportes trazables.
Grado de participación	Medio: participa en la definición de reglas y en las pruebas de aceptación.
Comentarios	Necesita reportes descargables para sus informes de auditoría.

Tabla 9

Perfil del Interesado: Equipo de Proyecto de Datastack

Descripción	Equipo proveedor responsable de construir la solución.
Tipo	Desarrollador.
Responsabilidades	Análisis, diseño, implementación, pruebas, despliegue, documentación y soporte.
Criterios de éxito	Entrega en plazo, aceptación del cliente y cumplimiento de los rangos de calidad.
Grado de participación	Total durante todo el proyecto.
Comentarios	Trabaja con iteraciones cortas y entregas incrementales.

3.5 Perfiles de los Usuarios

Las Tablas 10 a 13 describen el perfil de cada tipo de usuario del sistema.

Tabla 10
Perfil del Usuario: Administrador de Bases de Datos

Descripción	Profesional que administra los clústeres de MongoDB, Redis y Cassandra.
Tipo	Usuario experto.
Responsabilidades	Registrar y actualizar las conexiones, ejecutar diagnósticos y escaneos por lotes.
Criterios de éxito	Conexiones estables, diagnóstico del estado de cada motor y latencia visible.
Grado de participación	Uso frecuente (diario).
Comentarios	Utiliza principalmente los módulos Conexiones, Monitor y Ejecución.

Tabla 11
Perfil del Usuario: Analista de Calidad de Datos

Descripción	Profesional que traduce las políticas de negocio en reglas de integridad.
Tipo	Usuario intermedio.
Responsabilidades	Crear, revisar y eliminar reglas; analizar las anomalías detectadas.
Criterios de éxito	Definir reglas sin programar y obtener resultados comprensibles.
Grado de participación	Uso frecuente (varias veces por semana).
Comentarios	Utiliza los módulos Reglas, Ejecución y Reportes.

Tabla 12
Perfil del Usuario: Desarrollador de Aplicaciones

Descripción	Profesional que construye los servicios que escriben en los motores.
Tipo	Usuario experto.
Responsabilidades	Probar registros en el simulador previo a la escritura antes de desplegar cambios.
Criterios de éxito	Saber de inmediato si un registro cumple o incumple las reglas y por qué.
Grado de participación	Uso ocasional, ligado a cada cambio en las aplicaciones.
Comentarios	Utiliza el simulador del módulo Ejecución.

Tabla 13

Perfil del Usuario: Auditor Interno

Descripción	Profesional que supervisa el cumplimiento de las políticas de datos.
Tipo	Usuario básico.
Responsabilidades	Consultar métricas y descargar reportes de anomalías.
Criterios de éxito	Acceso a información clara, fechada y exportable.
Grado de participación	Uso periódico (mensual o por auditoría).
Comentarios	Utiliza los módulos Dashboard y Reportes.

3.6 Necesidades de los interesados y usuarios

La Tabla 14 relaciona cada necesidad identificada con su prioridad, la forma en que se atiende hoy y la solución que propone el producto.

Tabla 14

Necesidades de los Interesados y Usuarios

Necesidad	Prioridad	Inquietud	Solución actual	Solución propuesta
Validar la estructura de los registros	Alta	Documentos con campos faltantes o tipos incorrectos	Validaciones dispersas en el código	Reglas de esquema con JSON Schema y simulador previo a la escritura
Evitar registros duplicados	Alta	Clientes y pedidos duplicados	Consultas manuales	Reglas de unicidad por uno o varios campos
Asegurar referencias entre motores	Alta	Pagos asociados a pedidos inexistentes	Ninguna	Reglas de integridad referencial entre motores
Detectar réplicas divergentes	Media	Datos distintos para un mismo cliente	Revisión manual	Reglas de consistencia entre motores
Auditar las violaciones	Alta	Falta de evidencia ante auditorías	Hojas de cálculo	Historial de auditoría, tablero y reportes PDF
Proteger las credenciales	Alta	Contraseñas de motores en texto plano	Archivos locales	Cifrado Fernet por usuario y contraseñas con bcrypt
Conocer el estado de los motores	Media	Caídas no detectadas a tiempo	Consolas de cada proveedor	Módulo Monitor con latencia, versión y recursos

Nota. Elaboración propia.

4. Vista General del Producto

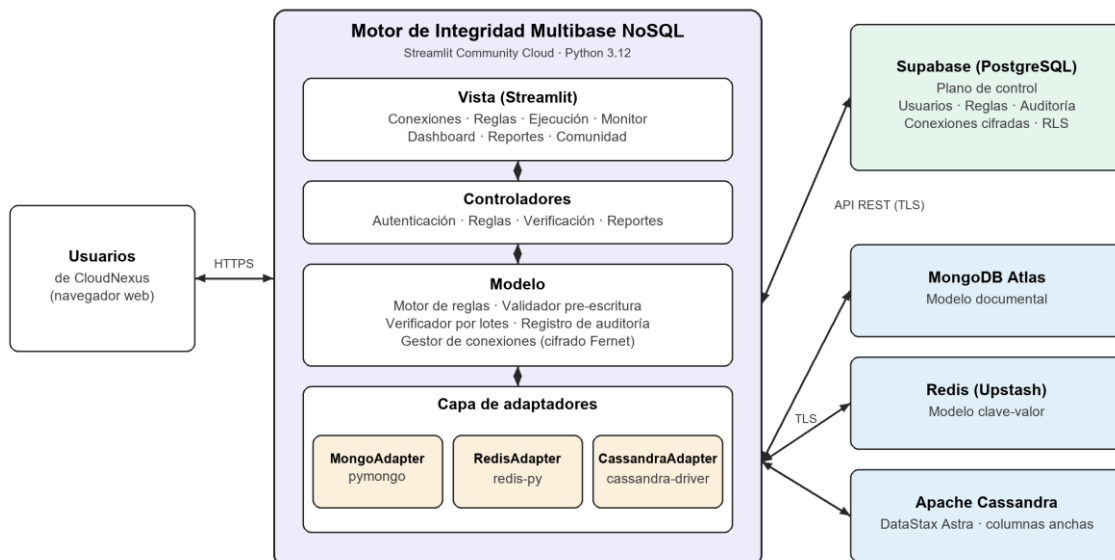
4.1 Perspectiva del producto

El Motor de Integridad Multibase NoSQL es un producto independiente que se ubica por encima de las bases de datos del cliente, sin reemplazarlas ni modificar las aplicaciones que las usan. Sigue el patrón Modelo-Vista-Controlador: la vista, construida con Streamlit, ofrece los módulos de trabajo; los controladores orquestan las peticiones; y el modelo contiene el motor de reglas, el validador previo a la escritura, el verificador por lotes, el registro de auditoría y el gestor de conexiones.

El acceso a cada motor se realiza mediante una capa de adaptadores con una interfaz común, lo que permite aplicar las mismas reglas a MongoDB, Redis y Cassandra y agregar nuevos motores en el futuro. Supabase actúa como plano de control y almacena los usuarios, las reglas, el historial de auditoría y las conexiones cifradas, protegidas con seguridad a nivel de fila. La Figura 1 muestra esta perspectiva.

Figura 1

Perspectiva del Producto y su Relación con los Sistemas Externos



Nota. Elaboración propia. Las comunicaciones con el usuario se realizan por HTTPS y las conexiones con Supabase y con los motores se cifran con TLS.

4.2 Resumen de capacidades

La Tabla 15 relaciona los beneficios que obtiene el cliente con las características del producto que los hacen posibles.

Tabla 15

Resumen de Capacidades

Beneficio para el cliente	Características que lo apoyan
Datos estructuralmente correctos	Reglas de esquema basadas en JSON Schema y simulador previo a la escritura.
Eliminación de duplicados	Reglas de unicidad y conteo de duplicados en cada motor.
Coherencia entre motores	Reglas de integridad referencial y de consistencia entre MongoDB, Redis y Cassandra.
Detección temprana de errores	Escaneo por lotes bajo demanda sobre los datos existentes.
Trazabilidad y cumplimiento	Historial de auditoría centralizado, tablero de métricas y reportes PDF con sugerencias.
Seguridad de la información	Credenciales cifradas con Fernet, contraseñas con bcrypt y RLS en el plano de control.
Visibilidad operativa	Monitor con estado de conexión, latencia, versión, esquemas y muestra de registros.
Acceso sin instalación	Aplicación web en la nube, accesible desde cualquier navegador moderno.

Nota. Elaboración propia.

4.3 Suposiciones y dependencias

El funcionamiento del producto se basa en las siguientes suposiciones:

- CloudNexus proporciona credenciales de acceso de lectura a sus bases de datos MongoDB, Redis y Cassandra.
- Los registros de Redis se almacenan como cadenas JSON o *hashes* agrupados por prefijo de clave (por ejemplo, *usuarios:* o *pedidos:*).
- Los usuarios cuentan con conexión estable a internet y un navegador actualizado.
- El volumen de datos de prueba se mantiene dentro de los límites de los planes gratuitos de los servicios en la nube.

Asimismo, el producto depende de los siguientes servicios y componentes externos:

- Streamlit Community Cloud para el despliegue de la aplicación y GitHub como repositorio del código fuente.
- Supabase como plano de control (PostgreSQL con RLS).
- MongoDB Atlas, Upstash (Redis) y DataStax Astra (Cassandra) como servicios gestionados de los motores.
- Las bibliotecas de Python pymongo, redis-py, cassandra-driver, supabase, jsonschema, cryptography, bcrypt, pandas y fpdf2.

4.4 Costos y precios

La Tabla 16 resume la inversión estimada para desarrollar la versión 1.0 durante cuatro meses, según el análisis económico del informe de factibilidad (FD01), donde se detalla cada concepto.

Tabla 16

Costos Estimados del Proyecto

Concepto	Detalle	Costo (S/)
Costos generales	Depreciación de equipos, útiles, impresiones y memoria USB	801.67
Costos operativos	Internet, energía eléctrica, telefonía y movilidad durante cuatro meses	1 320.00
Costos del ambiente	Servicios en la nube y herramientas en planes gratuitos (Tabla 17)	0.00
Costos de personal	Dos integrantes, 320 horas cada uno a S/ 15.00 por hora	9 600.00
Total		11 721.67

Nota. Elaboración propia con base en el informe de factibilidad (FD01).

Durante el desarrollo se emplean los planes gratuitos de los proveedores en la nube, cuyos límites se muestran en la Tabla 17. Si el volumen de datos de CloudNexus supera esos límites, los planes de pago corren por cuenta del cliente; por ejemplo, el plan Pro de Supabase parte de 25 dólares mensuales (Supabase Inc., s.f.).

Tabla 17
Servicios en la Nube Utilizados

Servicio	Uso en el producto	Plan y límite principal
MongoDB Atlas	Motor documental	M0 gratuito: hasta 512 MB de almacenamiento (MongoDB, Inc., s.f.-a)
Upstash	Motor clave-valor (Redis)	Gratuito: 256 MB y 500 000 comandos al mes (Upstash, Inc., s.f.)
DataStax Astra	Motor de columnas anchas (Cassandra)	Plan gratuito con créditos mensuales
Supabase	Plano de control	Gratuito: base de datos de 500 MB por proyecto (Supabase Inc., s.f.)
Streamlit Community Cloud	Despliegue de la aplicación	Gratuito para aplicaciones públicas (Snowflake Inc., s.f.)

Nota. Elaboración propia.

Con base en estos costos, Datastack propone a CloudNexus un precio de implementación de S/ 14 000.00, que incluye el despliegue, la configuración inicial de reglas y la capacitación de los usuarios, y un servicio opcional de soporte y mantenimiento de S/ 350.00 mensuales. El análisis financiero del FD01 muestra que la inversión es rentable para el cliente, con un VAN de S/ 9 057.58 y una TIR de 46.97 %.

4.5 Licenciamiento e instalación

El código fuente del producto es propiedad de Datastack y se entrega a CloudNexus bajo una licencia de uso no exclusiva, conforme a la Ley sobre el Derecho de Autor (Presidencia de la República del Perú, 1996). Todas las dependencias son de código abierto y sus licencias permiten el uso comercial, como se muestra en la Tabla 18.

Tabla 18

Dependencias y Licencias

Componente	Función	Licencia
Streamlit	Interfaz web	Apache 2.0
pymongo	Conexión con MongoDB	Apache 2.0
redis-py	Conexión con Redis	MIT
cassandra-driver	Conexión con Cassandra	Apache 2.0
supabase-py	Conexión con el plano de control	MIT
jsonschema	Validación de esquemas	MIT
cryptography	Cifrado Fernet de credenciales	Apache 2.0 o BSD
bcrypt	Protección de contraseñas	Apache 2.0
fpdf2	Generación de reportes PDF	LGPL 3.0
pandas	Manejo de datos tabulares	BSD 3-Clause
pytest	Pruebas unitarias	MIT

Nota. Elaboración propia.

El producto se ofrece como aplicación web, por lo que el usuario final no instala nada. Para desplegar una nueva instancia, el equipo técnico sigue estos pasos:

1. Crear un proyecto en Supabase y ejecutar el script *supabase_schema.sql* para generar las tablas.
2. Generar la clave de cifrado (ENCRYPTION_KEY) y registrar SUPABASE_URL, SUPABASE_KEY y ENCRYPTION_KEY como secretos de la aplicación.
3. Publicar el repositorio en Streamlit Community Cloud con *vista/app.py* como archivo principal y Python 3.12.
4. Configurar los servicios de MongoDB Atlas, Upstash y DataStax Astra para aceptar conexiones desde la aplicación.
5. Iniciar sesión, registrar las conexiones a los motores y definir las reglas de integridad.

5. Características del producto

La Tabla 19 describe las características que ofrece el producto en su versión 1.0.

Tabla 19

Características del Producto

Código	Característica	Descripción
CP01	Autenticación de usuarios	Registro e inicio de sesión con contraseñas protegidas con bcrypt; cada usuario tiene un espacio de trabajo propio.
CP02	Gestión de conexiones	Registro, prueba y eliminación de conexiones a MongoDB, Redis y Cassandra (incluido el <i>Secure Connect Bundle</i> de Astra), cifradas con Fernet.
CP03	Gestión de reglas	Creación, consulta y eliminación de reglas de esquema, unicidad, integridad referencial y consistencia, con motor de origen y de destino.
CP04	Simulador previo a la escritura	Valida un registro JSON contra las reglas activas antes de escribirlo e informa cada incumplimiento.
CP05	Escaneo por lotes	Recorre los datos existentes de los motores y registra cada violación detectada en el historial de auditoría.
CP06	Monitor y observabilidad	Muestra el estado de conexión, la latencia, la versión del motor, los esquemas, una muestra de registros y las estadísticas del servidor.
CP07	Dashboard	Presenta métricas de motores utilizados, reglas aplicadas y anomalías detectadas.
CP08	Reportes de anomalías	Lista las violaciones con sugerencias de corrección y permite descargarlas en PDF.
CP09	Comunidad	Muestra los usuarios conectados en el sistema.
CP10	Auditoría centralizada	Conserva en Supabase el historial de violaciones con regla, motor, fecha y dato afectado.

Nota. Elaboración propia.

6. Restricciones

El producto está sujeto a las siguientes restricciones:

- Solo admite los motores MongoDB, Redis y Apache Cassandra en su versión 1.0.
- Requiere acceso a internet; no existe un modo de trabajo sin conexión.
- La capacidad está limitada por los planes gratuitos de los servicios en la nube mientras el cliente no contrate planes de pago.
- El escaneo por lotes se ejecuta bajo demanda; no se programa de forma automática.

- El sistema detecta y reporta violaciones, pero no corrige los datos ni bloquea escrituras de aplicaciones de terceros.
- La aplicación debe desarrollarse con Python y Streamlit, y desplegarse en Streamlit Community Cloud.
- El proyecto debe concluir dentro del semestre académico 2026-II.

7. Rangos de calidad

Los rangos de calidad se definen a partir del modelo de calidad de producto de la norma ISO/IEC 25010:2023 (International Organization for Standardization [ISO], 2023), como se detalla en la Tabla 20.

Tabla 20

Rangos de Calidad del Producto

Característica	Rango esperado	Forma de medición
Adecuación funcional	100 % de las violaciones del conjunto de prueba detectadas y 0 % de falsos positivos	Escaneo por lotes sobre datos con violaciones inyectadas
Eficiencia de desempeño	Validación individual en menos de 3 s y latencia de diagnóstico mostrada por motor	Medición en el simulador y en el módulo Monitor
Compatibilidad	Operación simultánea con los tres motores y con el plano de control	Pruebas de conexión en el módulo Monitor
Capacidad de interacción	Un usuario nuevo crea una regla y ejecuta un escaneo en menos de 10 minutos	Prueba con usuarios de CloudNexus
Fiabilidad	Disponibilidad sujeta a la del proveedor; los errores de conexión se informan sin detener la aplicación	Pruebas de fallos de conexión
Seguridad	100 % de credenciales cifradas y 0 credenciales en el repositorio	Revisión del plano de control y del repositorio
Mantenibilidad	100 % de pruebas unitarias aprobadas y arquitectura en capas con adaptadores	Ejecución de pytest
Flexibilidad	Incorporación de un nuevo motor implementando un adaptador, sin cambiar el motor de reglas	Revisión de la arquitectura

Nota. Elaboración propia.

8. Precedencia y Prioridad

La Tabla 21 establece la prioridad de cada característica y la iteración en la que se construye. Las características de prioridad alta conforman el producto mínimo y son requisito de las demás.

Tabla 21

Precedencia y Prioridad de las Características

Código	Característica	Prioridad	Iteración
CP01	Autenticación de usuarios	Alta	1
CP02	Gestión de conexiones	Alta	1
CP03	Gestión de reglas	Alta	2
CP04	Simulador previo a la escritura	Alta	2
CP05	Escaneo por lotes	Alta	2
CP10	Auditoría centralizada	Alta	2
CP07	Dashboard	Media	3
CP08	Reportes de anomalías	Media	3
CP06	Monitor y observabilidad	Media	3
CP09	Comunidad	Baja	3

Nota. Elaboración propia.

9. Otros requerimientos del producto

Además de sus características funcionales, el producto debe cumplir los estándares que se describen a continuación.

a) Estándares legales

- Ley N.° 29733, Ley de Protección de Datos Personales (Congreso de la República del Perú, 2011), y su reglamento aprobado por el Decreto Supremo N.° 016-2024-JUS (Ministerio de Justicia y Derechos Humanos, 2024), que exigen medidas técnicas y organizativas para proteger los datos personales.
- Ley N.° 30096, Ley de Delitos Informáticos (Congreso de la República del Perú, 2013), que sanciona el acceso ilícito a sistemas y datos.
- Decreto Legislativo N.° 822, Ley sobre el Derecho de Autor (Presidencia de la República del Perú, 1996), que protege el software como obra.

- Cumplimiento de las licencias de código abierto de cada dependencia (Tabla 18).

b) Estándares de comunicación

- HTTPS entre el navegador y la aplicación, conforme a la semántica de HTTP (Fielding et al., 2022).
- Cifrado TLS en las conexiones con Supabase, MongoDB Atlas, Upstash y DataStax Astra (Rescorla, 2018).
- Intercambio de datos y definición de esquemas en formato JSON (Bray, 2017) y JSON Schema (JSON Schema, 2022).
- Mensajes de la interfaz en español, claros y orientados a la acción.

c) Estándares de cumplimiento de la plataforma

- Python 3.12 y las versiones mínimas de dependencias declaradas en *requirements.txt*.
- Despliegue en Streamlit Community Cloud y compatibilidad con las versiones actuales de Chrome, Edge y Firefox.
- Código fuente conforme a la guía de estilo PEP 8 (van Rossum et al., 2001) y organizado según el patrón MVC.
- Control de versiones en GitHub con ramas e historial de cambios.

d) Estándares de calidad y seguridad

- Modelo de calidad de producto ISO/IEC 25010:2023 (ISO, 2023), aplicado en los rangos del capítulo 7.
- Buenas prácticas de seguridad de la información basadas en ISO/IEC 27001:2022 (ISO, 2022).
- Prevención de los riesgos del OWASP Top 10 (OWASP Foundation, 2021), en especial el control de acceso, las fallas criptográficas y la inyección.
- Contraseñas con bcrypt (Provos y Mazières, 1999), credenciales cifradas con Fernet (Python Cryptographic Authority, s.f.) y seguridad a nivel de fila en Supabase (Supabase Inc., s.f.).
- Pruebas unitarias automatizadas con pytest antes de cada publicación.

CONCLUSIONES

El Motor de Integridad Multibase NoSQL responde a una necesidad concreta de CloudNexus: asegurar la calidad de datos distribuidos en motores que no imponen esquema, unicidad, integridad referencial ni consistencia entre ellos.

La identificación de cuatro interesados y cuatro tipos de usuario permitió priorizar las capacidades del producto: las reglas y su verificación forman el núcleo, mientras que el tablero, los reportes y el monitor aportan trazabilidad y visibilidad.

La arquitectura en capas con adaptadores y un plano de control en Supabase permite aplicar las mismas reglas a MongoDB, Redis y Cassandra y facilita la incorporación de nuevos motores.

El uso de servicios en la nube con planes gratuitos y de bibliotecas de código abierto mantiene un costo de desarrollo de S/ 11 721.67 y un precio competitivo de S/ 14 000.00 para el cliente.

RECOMENDACIONES

- Validar con el Área de Calidad de Datos de CloudNexus el catálogo inicial de reglas antes de las pruebas de aceptación.
- Evaluar la programación automática de escaneos por lotes y el envío de alertas en una versión posterior.
- Migrar a planes de pago de los servicios en la nube cuando el volumen de datos supere los límites gratuitos.
- Rotar periódicamente las credenciales de los motores y la clave de cifrado siguiendo un procedimiento controlado.
- Utilizar este documento como base para la especificación de requisitos (FD03) y la arquitectura (FD04).

BIBLIOGRAFIA

- Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8259>
- Congreso de la República del Perú. (2011, 3 de julio). Ley N.º 29733, Ley de Protección de Datos Personales. *Diario Oficial El Peruano*.
- Congreso de la República del Perú. (2013, 22 de octubre). Ley N.º 30096, Ley de Delitos Informáticos. *Diario Oficial El Peruano*.
- Fielding, R., Nottingham, M., y Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9110>
- Gilbert, S., y Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59. <https://doi.org/10.1145/564585.564601>
- International Organization for Standardization. (2022). *Information security, cybersecurity and privacy protection — Information security management systems — Requirements* (ISO/IEC Standard No. 27001:2022).
- International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC Standard No. 25010:2023).
- Kruchten, P. (2004). *The Rational Unified Process: An introduction* (3.ª ed.). Addison-Wesley.
- Presidencia de la República del Perú. (1996, 24 de abril). Decreto Legislativo N.º 822, Ley sobre el Derecho de Autor. *Diario Oficial El Peruano*.
- Provos, N., y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference* (pp. 81–91). USENIX Association.
- Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8446>

Sadalage, P. J., y Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44.

<https://doi.org/10.1145/1435417.1435432>

WEBGRAFIA

The Apache Software Foundation. (s.f.). *Introduction* [Modelado de datos]. Apache

Cassandra Documentation. Recuperado el 3 de octubre de 2026, de

<https://cassandra.apache.org/doc/latest/cassandra/developing/data-modeling/intro.html>

JSON Schema. (2022). *JSON Schema draft 2020-12*. <https://json-schema.org/draft/2020-12>

Ministerio de Justicia y Derechos Humanos. (2024, 30 de noviembre). *Decreto Supremo N.º*

016-2024-JUS que aprueba el Reglamento de la Ley N.º 29733, Ley de Protección de Datos Personales. https://www.congreso.gob.pe/Docs/DGP/DIDP/files/ds_016-2024-jus.pdf

MongoDB, Inc. (s.f.-a). *Atlas M0 (free cluster) limits*. MongoDB Docs. Recuperado el 3 de

octubre de 2026, de <https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/>

MongoDB, Inc. (s.f.-b). *Schema validation*. MongoDB Docs. Recuperado el 3 de octubre de

2026, de <https://www.mongodb.com/docs/manual/core/schema-validation/>

OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>

Python Cryptographic Authority. (s.f.). *Fernet (symmetric encryption)*. Cryptography

documentation. Recuperado el 3 de octubre de 2026, de

<https://cryptography.io/en/latest/fernet/>

Redis Ltd. (s.f.). *Redis data types*. Redis Docs. Recuperado el 3 de octubre de 2026, de

<https://redis.io/docs/latest/develop/data-types/>

van Rossum, G., Warsaw, B., y Coghlan, A. (2001). *PEP 8 – Style guide for Python code*.

Python Software Foundation. <https://peps.python.org/pep-0008/>

Sakpal, M. (2021, 14 de julio). *How to improve your data quality*. Gartner.

<https://www.gartner.com/smarterwithgartner/how-to-improve-your-data-quality>

Snowflake Inc. (s.f.). *Streamlit Community Cloud*. Streamlit Docs. Recuperado el 3 de

octubre de 2026, de <https://docs.streamlit.io/deploy/streamlit-community-cloud>

Stack Overflow. (2025). *2025 Stack Overflow Developer Survey: Technology*.

<https://survey.stackoverflow.co/2025/technology>

Supabase Inc. (s.f.). *Pricing*. Recuperado el 3 de octubre de 2026, de

<https://supabase.com/pricing>

Upstash, Inc. (s.f.). *Upstash Redis pricing*. Recuperado el 3 de octubre de 2026, de

<https://upstash.com/pricing/redis>